

DCST1005 – Digital infrastruktur og cybersikkerhet

Mock Eksamen 1 – Løsningsforslag

Del A – Flervalg (fasit)

Spørsmål	Korrekt svar
1	b
2	b
3	b
4	a
5	d
6	b
7	b
8	b
9	b
10	b
11	c
12	c

Del B – Windows Server og on-premises administrasjon

Oppgave B1 – Brukere og tilgangsstyring

a) Klargjøring av AD-miljøet for Prosjekt-avdelingen

Følgende steg utføres for å klargjøre AD-miljøet:

1. Opprett en ny OU kalt **Prosjekt** under **OU=Users,OU=InfraIT,DC=infraIT,DC=sec** for brukerkontoene, og en tilsvarende OU under **OU=Grupper,OU=InfraIT** for gruppene.
2. Opprett en **Global Security Group** kalt **g_Prosjekt**. Alle ansatte i avdelingen legges inn som medlemmer av denne gruppen.

3. Opprett en Domain Local Security Group kalt `l_Prosjekt-Les` og/eller `l_Prosjekt-Skriv` avhengig av hvilke tilgangsnivåer som trengs.
4. Legg `g_Prosjekt` inn i den aktuelle `l_gruppen` i henhold til AGDLP-prinsippet.
5. Opprett prosjektmappen på `SRV1` og del den ut som en nettverksressurs.
6. Tildel NTFS-tillatelser på mappen til `l_Prosjekt-Skriv` (Modify) og eventuelt `l_Prosjekt-Les` (Read & Execute).

Begrunnelse for strukturen: Ved å følge AGDLP-prinsippet holdes tilgangsstyringen fleksibel. Dersom en bruker bytter avdeling, endres kun gruppemedlemskapet – ikke tillatelsene på selve ressursen. Domain Local-grupper knyttes til ressursen, globale grupper organiserer brukerne.

b) Identifisering og håndtering av inaktive brukerkontoer

Inaktive kontoer kan identifiseres ved å bruke PowerShell-cmdleten `Search-ADAccount -AccountInactive -TimeSpan 90.00:00:00`, som returnerer alle kontoer som ikke har vært i bruk på over 90 dager. Alternativt kan `Get-ADUser` med filter på `LastLogonDate` brukes.

Anbefalt fremgangsmåte:

1. Eksporter listen over inaktive kontoer for gjennomgang
2. **Deaktiver** kontoene fremfor å slette dem umiddelbart – dette gir mulighet til å reaktivere dersom en bruker er på permisjon eller sykmeldt
3. Flytt deaktiverte kontoer til en egen OU kalt `Deaktivert` for oversiktighetens skyld
4. Logg alle endringer med tidsstempel og ansvarlig administrator
5. Etter en definert karanteneperiode (f.eks. 30–90 dager) kan kontoene slettes permanent

Sikkerhetsrisiko ved inaktive kontoer: En gammel, aktiv brukerkonto er et potensielt angrepspunkt. En trusselaktør som får tak i legitimasjonen til en inaktiv konto kan bruke den til å bevege seg lateralt i nettverket uten at det nødvendigvis utløser alarmer, siden kontoen ikke er i daglig bruk og avvikende aktivitet kan gå ubemerket hen.

c) Delegering av passordtilbakestilling til helpdesk

I Active Directory kan administrasjon delegeres på OU-nivå uten å gi fullstendige administratorrettigheter. For å gi helpdesk-ansatte rettighet til kun å tilbakestille passord i Prosjekt-OU-en:

1. Høyreklikk på `OU=Prosjekt` i Active Directory Users and Computers
2. Velg «**Delegate Control**»
3. Legg til helpdesk-brukeren eller en sikkerhetsgruppe for helpdesk (f.eks. `g_Helpdesk`)

4. Velg oppgaven «Reset user passwords and force password change at next logon»

Resultatet er at helpdesk kan tilbakestille passord for brukere i akkurat denne OU-en, men har ingen rettigheter til å opprette, slette eller endre andre egenskaper ved brukerobjektene, og ingen rettigheter i andre deler av AD.

Oppgave B2 – Group Policy

a) BitLocker-kryptering via GPO

BitLocker konfigureres under `Computer Configuration → Administrative Templates → Windows Components → BitLocker Drive Encryption`. Relevante innstillinger:

- «**Require additional authentication at startup**» – aktiveres for å styre oppstartsautentisering
- «**Choose how BitLocker-protected operating system drives can be recovered**» – her aktiveres lagring av gjenopprettingsnøkkel i Active Directory ved å huke av «**Save BitLocker recovery information to AD DS**» og sette «**Do not enable BitLocker until recovery information is stored to AD DS**»

GPO-en kobles til OU-en `InfraIT > Datamaskiner > Klienter` slik at den kun treffer bærbare PC-er og ikke servere.

For at BitLocker faktisk skal aktiveres på maskinene må TPM-brikken være til stede og aktiv, og maskinene må ha blitt med i domenet og mottatt GPO-en. Selve krypteringen kan ytterligere håndheves via et oppstartscript som kjører `manage-bde -on C:` dersom BitLocker ikke allerede er aktivt.

b) To måter å unnta IT-drift fra USB-blokkeringspolicyen

Alternativ 1 – Security filtering (filtrering på sikkerhetsgruppe): GPO-er gjelder som standard for alle autentiserte brukere i OU-en den er koblet til. Ved å fjerne gruppen «Authenticated Users» fra GPO-ens sikkerhetsfiltrering og legge til en gruppe kalt f.eks. `g_USB-Blokkert` (alle unntatt IT-drift), vil IT-drift ikke få policyen anvendt.

Fordeel: Enkel å administrere – man legger bare brukere i eller ut av gruppen.

Ulempe: Krever at man holder gruppene oppdatert, og kan være forvirrende dersom mange unntak bygger seg opp over tid.

Alternativ 2 – Block Inheritance på IT-drift sin OU: Sett **Block Inheritance** på `OU=IT-drift` slik at domenepolicyen som blokkerer USB ikke arves ned til denne OU-en.

Fordeel: Ryddig og tydelig – IT-drift-OU-en er eksplisitt skjermet fra overordnede policyer. *Ulempe:* Block Inheritance blokkerer **alle** nedarvede GPO-er, ikke bare USB-policyen. Dette kan utilsiktet fjerne andre ønskede policyer fra IT-drift-brukerne, noe som krever at disse rekonfigureres eksplisitt på OU-nivå. Bruk Enforced på de GPO-ene som skal gjelde IT-drift uansett.

Enforced overstyrer Block Inheritance, så dersom du setter Enforced på f.eks. sikkerhetsbaseline-GPO-en på domenenivå, vil den trenge gjennom selv om IT-drift-OU-en har Block Inheritance. USB-GPO-en forblir uten Enforced og blokkeres dermed. Dette er en brukbar tilnærming, men kan bli komplisert å vedlikeholde dersom mange GPO-er trenger Enforced.

c) Loopback processing i Group Policy

Normalt bestemmes hvilke User Configuration-innstillinger en bruker får av hvilken OU brukerkontoen befinner seg i. Loopback processing endrer denne logikken slik at User Configuration-innstillingene bestemmes av hvilken maskin brukeren logger på, ikke av brukerens OU-plassering.

Det finnes to modi:

- **Replace:** Kun maskinens GPO-baserte brukerinnstillinger gjelder – brukerens egne GPO-er ignoreres
- **Merge:** Maskinens GPO-baserte brukerinnstillinger kombineres med brukerens egne, der maskinens innstillinger har høyest prioritet ved konflikt

Eksempel på bruk: Et kioskmiljø eller en Terminal Server der mange ulike brukere logger inn på samme maskin. Uavhengig av hvem som logger på skal alle brukere ha den samme, låste brukeropplevelsen (f.eks. ingen tilgang til kontrollpanel, fast skrivebordsbakgrunn, begrenset programtilgang). Med loopback processing konfigurert på maskin-OU-en vil disse restriksjonene alltid gjelde på den aktuelle maskinen, uavhengig av brukerens vanlige GPO-innstillinger.

Del C – Azure og offentlig sky

Oppgave C1 – Nettverkssikkerhet

a) Defense in depth med NSG og Azure Firewall

Defense in depth er et sikkerhetsprinsipp som innebærer å bruke flere lag med sikkerhetskontroller, slik at en angriper må bryte gjennom flere barrierer for å nå en ressurs. Dersom ett lag svikter, stoppes angriperen av neste lag.

I en hub-spoke topologi kan dette realiseres slik:

- **Ytterste lag – Azure Firewall i hub-en:** All trafikk inn og ut av spoke-ene tvinges gjennom Azure Firewall via UDR-er. Azure Firewall opererer på lag 7 og kan filtrere basert på FQDN, applikasjonsregler og trusselintelligens. Den gir sentralisert logging og oversikt over all trafikk på tvers av spoke-ene.
- **Innerste lag – NSG på subnett og NIC-nivå:** NSG-er i hvert spoke-subnett fungerer som det siste forsvarslinjen og filtrerer trafikk på lag 3/4 basert på IP og port. Selv om Azure Firewall tillater trafikk, kan NSG-en blokkere den dersom den ikke oppfyller subnettets egne regler.

Kombinasjonen gir to uavhengige kontrollpunkter: Azure Firewall for avansert, sentralisert inspeksjon og NSG for granulær, ressursnær kontroll.

b) NSG-regler for isolert VM-til-database-kommunikasjon

For å tillate at en applikasjons-VM kun kan kommunisere med en bestemt database-VM, og blokkere all annen utgående trafikk, opprettes følgende regler på NSG-en tilknyttet applikasjons-VM-ens subnett:

Prioritet	Navn	Retning	Kilde	Destinasjon	Port	Protokoll
100	Tillat-til-DB	Utgående	*	<DB-VM privat IP>	1433 (eller aktuell DB- port)	TCP
4096	Blokker-alt-ut	Utgående	*	*	*	*

NSG-er har en innebygd standardregel `AllowVnetOutBound` med prioritet 65000 som tillater all utgående VNet-trafikk. Denne overstyres av den eksplisitte Deny-regelen på prioritet 4096, som blokkerer alt annet utgående. Allow-regelen på prioritet 100 sørger for at database-trafikken passerer før Deny-regelen evalueres.

c) Stateful trafikkinspeksjon

Stateful trafikkinspeksjon betyr at brannmuren eller sikkerhetskomponenten holder rede på aktive tilkoblinger og automatisk tillater returtrafikk for etablerte forbindelser, uten at man trenger å opprette eksplisitte regler for begge retninger.

NSG-er i Azure er stateful. Dette betyr at dersom man oppretter en inngående Allow-regel for TCP port 22 (SSH), vil svar-trafikken fra serveren tilbake til klienten automatisk tillates – selv uten en eksplisitt utgående regel for port 22.

Azure holder styr på at denne trafikken er en del av en etablert tilkobling.

I praksis betyr dette at man normalt kun trenger å definere regler for én retning av en tilkobling. Det forenkler NSG-konfigurasjonen betydelig og reduserer risikoen for at feilkonfigurerte bidireksjonale regler åpner for utilsiktet trafikk.

Oppgave C2 – Tilgangsstyring og kostnadsbevissthet

a) Lesetilgang til abonnement med RBAC

Den innebygde rollen **Reader** passer her. Reader gir lesetilgang til alle ressurser i det valgte scope, men ingen mulighet til å opprette, endre eller slette ressurser.

Slik tildeles rollen:

1. Naviger til det aktuelle abonnementet i Azure Portal

2. Velg **Access control (IAM)**
3. Klikk **Add → Add role assignment**
4. Velg rollen **Reader**
5. Under «Members», søk opp og velg den aktuelle brukeren
6. Klikk **Review + assign**

Ved å tildele rollen på abonnementsnivå (scope = subscription) gjelder tilgangen automatisk for alle ressursgrupper og ressurser innenfor abonnementet, uten at man må gjenta tildelingen for hver enkelt ressursgruppe.

b) Azure Policy og forskjellen fra RBAC

Azure Policy er et verktøy for å håndheve organisasjonsstandarter og vurdere samsvar på tvers av Azure-ressurser. Der RBAC styrer **hvem som kan gjøre hva**, styrer Azure Policy **hva som er lov å gjøre** – uavhengig av hvem som gjør det.

For å håndheve at alle virtuelle maskiner får taggen **Miljø: Produksjon** :

1. Opprett en policy med effekten «**Modify**» eller «**Append**»
2. Definer at alle ressurser av typen **Microsoft.Compute/virtualMachines** skal ha taggen **Miljø** med verdien **Produksjon**
3. Tilordne policyen til det aktuelle abonnementet eller ressursgruppen
4. Med effekten **Modify** vil Azure automatisk legge til taggen på eksisterende og nye ressurser som mangler den

Forskjellen fra RBAC: RBAC handler om tilgangskontroll – det begrenser hva en bruker kan utføre. Azure Policy handler om ressurskontroll – det styrer hvordan ressurser skal konfigureres og kan hindre opprettelse av ressurser som ikke er i samsvar, uavhengig av brukerens rolle. En bruker med Contributor-rollen kan opprette ressurser fritt, men Azure Policy kan likevel nekte opprettelsen dersom ressursen ikke oppfyller policy-kravene.

c) Tre tiltak for å identifisere og redusere uventede Azure-kostnader

Tiltak 1 – Gjennomgå Cost Analysis i Azure Portal: Under «Cost Management + Billing» kan man analysere kostnader fordelt på ressursgruppe, ressurstype og tidsperiode. Dette avslører raskt hvilke ressurser eller tjenester som står for den uventede økningen. Særlig bør man se etter ressurser som kjører kontinuerlig uten behov, som virtuelle maskiner som burde vært stoppet utenom arbeidstid.

Tiltak 2 – Se etter «orphaned resources» (foreldreløse ressurser):

Ressurser som disk, offentlige IP-adresser og nettverksgrensesnitt fortsetter å faktureres selv etter at den tilknyttede virtuelle maskinen er slettet. En gjennomgang av ressursgrupper for å identifisere og slette slike foreldreløse ressurser kan gi umiddelbar kostnadsreduksjon.

Tiltak 3 – Sett opp budsjettvarsel: Opprett et budsjett i Cost Management med varsler som sendes på e-post når forbruket når 80 % og 100 % av budsjettet. Dette gir proaktiv kontroll og hindrer at lignende overraskelser skjer igjen fremover.

Del D – PowerShell og automatisering

Oppgave D1a – Pseudokode - MERK! Her er det logikken i scriptene som er viktig å kunne, ikke selve Verb-Substantivene. En kan skrive hva en vil, f.eks: Hent-Bruker, Les-CSV, Flytt-bruker, Slett-Bruker etc. En bør derimot kjenne til hvordan foreach, try-catch, if, if-else fungerer for å plassere det inn i pseudokode. Se to versjoner av løsningsforslag.

Forslag 1:

```
# Definer stier og variabler
$brukerlisteSti = "C:\Scripts\inaktive-brukere.txt"
$loggSti        = "C:\Scripts\deaktivering-logg.txt"
$deaktivertOU   = "OU=Deaktivert,OU=InfraIT,DC=infrait,DC=sec"

# Funksjon for logging med tidsstempel
function Skriv-Logg {
    param($Melding)
    $tidsstempel = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    "$tidsstempel - $Melding" | Add-Content -Path $loggSti
}

Skriv-Logg "--- Deaktivering av inaktive brukere startet ---"

# Les inn liste over inaktive brukere
$brukere = Get-Content -Path $brukerlisteSti

# Iterer over hver bruker i listen
foreach ($brukernavn in $brukere) {

    # Hopp over tomme linjer
    if ([string]::IsNullOrEmpty($brukernavn)) {
        continue
    }

    try {
        # Hent brukerobjektet fra AD
        $bruker = Get-ADUser -Identity $brukernavn -ErrorAction Stop

        # Deaktiver brukerkontoen
        Disable-ADAccount -Identity $brukernavn -ErrorAction Stop
        Skriv-Logg "SUKSESS: $brukernavn er deaktivert"
```

```

        # Flytt brukerkontoen til Deaktivert-OU
        Move-ADObject -Identity $bruker.DistinguishedName -TargetPat
        Skriv-Logg "SUKSESS: $brukernavn flyttet til $deaktivertOU"
    }
    catch {
        Skriv-Logg "FEIL: Kunne ikke behandle $brukernavn - $_"
    }
}

Skriv-Logg "--- Deaktivering fullført ---"

```

Forslag 2:

```

# VARIABLER
brukerliste = "sti til tekstfil med inaktive brukernavn"
loggfil      = "sti til loggfil"
deaktivertOU = "sti til Deaktivert-OU i AD"

# FUNKSJON: Skriv melding med tidsstempel til loggfil
FUNKSJON Skriv-Logg(melding):
    tidsstempel = Hent-NåværendeTidspunkt()
    Skriv-TilFil(loggfil, "$tidsstempel - $melding")

# START
Skriv-Logg("--- Deaktivering av inaktive brukere startet ---")

# Les inn alle brukernavn fra fil
brukere = Les-Tekstfil(brukerliste)

# Gå gjennom hver bruker i listen
FOR HVER brukernavn I brukere:

    # Hopp over tomme linjer
    HVIS brukernavn er tom eller bare mellomrom:
        FORTSETT til neste

    FORSØK:
        # Slå opp brukeren i AD
        bruker = Hent-ADBruker(brukernavn)

        # Deaktiver kontoen
        Deaktiver-ADKonto(brukernavn)
        Skriv-Logg("SUKSESS: $brukernavn er deaktivert")

        # Flytt kontoen til Deaktivert-OU
        Flytt-ADObjekt(bruker.DistinguishedName, deaktivertOU)
        Skriv-Logg("SUKSESS: $brukernavn flyttet til $deaktivertOU")

    VED FEIL:
        Skriv-Logg("FEIL: Kunne ikke behandle $brukernavn - feilmeld

```

```
# SLUTT
```

```
Skriv-Logg("--- Deaktivering fullført ---")
```

Oppgave D1b – Begrunnelser

Valg 1 – Try-Catch rundt AD-operasjonene

Uten feilhåndtering vil scriptet stoppe helt dersom én bruker ikke finnes i AD, for eksempel fordi brukernavnet i tekstfilen har en skrivefeil. Med Try-Catch fanges feilen opp, logges med en beskrivende melding, og scriptet fortsetter med neste bruker. I et produksjonsmiljø der listen kan inneholde mange brukere er det uakseptabelt at én feil stopper hele prosessen og etterlater resten av brukerne ubehandlet.

Valg 2 – Logging med tidsstempel til fil

Deaktivering av brukerkontoer er en sensitiv operasjon som bør være sporbar. En loggfil med tidsstempel dokumenterer nøyaktig hvilke kontoer som ble deaktivert og når, samt hvilke som eventuelt feilet og hvorfor. Dette er verdifullt både ved feilsøking og ved revisjon – for eksempel dersom noen i etterkant spør hvorfor en bestemt konto ble deaktivert.

Valg 3 – Sjekk på tomme linjer med if-blokk

En tekstfil kan inneholde tomme linjer, særlig på slutten av filen. Uten en eksplisitt sjekk vil Get-ADUser forsøke å slå opp en bruker med et tomt brukernavn, noe som vil generere en unødvendig feilmelding i loggen og kan forvirre den som leser rapporten. Ved å hoppe over tomme linjer med continue holdes loggen ren og scriptet mer forutsigbart.